

Simulation of the Central Limit Theorem

You are given a long list of i.i.d. samples drawn from an unknown distribution. Your task is to simulate the Central Limit Theorem (CLT) by grouping these samples into batches of size N .

Task

Implement a function named `student_compute_sample_means`. This function takes two arguments:

- `data`: a 1D NumPy array of i.i.d. samples of length
-
- `N`: a positive integer specifying how many data points to use when computing each sample mean.

The function should return a NumPy array

of

values that demonstrate the Central Limit Theorem. For each

(

) the values of the chunk

of data with

entries should be taken into account.

Use only complete chunks of size

. If the last chunk is smaller than

, discard it. Also, use only the chunks `data[0:N]`, `data[N:2*N]`, and so on.

For large N , these return values

should approximate samples from the standard normal distribution

Hint: The CLT requires that the mean and variance of the random variables are known. Because this information is not available to you, you need to estimate the mean and standard deviation from all samples using `data.mean()` and `data.std()`. The square root of N can be computed with `np.sqrt(N)`.

 [Click here to reveal/hide background information on the KL Divergence](#)

KL Divergence

To evaluate how well the Central Limit Theorem holds, we use the Kullback-Leibler (KL) divergence. KL divergence measures how different two probability distributions are. In this case, we compare the empirical distribution of your standardized sample means to the standard normal distribution.

A KL divergence close to 0 means the two distributions are very similar.

A higher KL divergence indicates more deviation from normality.

Goal: For large N, your sample means (after standardization) should result in a small KL divergence when compared to the standard normal distribution.

Note that you do not need to compute the KL distance, this is done by us based on your computed sample means.

Input / Output examples

Expected at the command line are

- the distribution (currently one of "uniform", "exponential", "poisson")
- the number N

Example Input

```
Enter <distribution> <N>:  
uniform 3
```

Example Output

```
KL divergence: 0.0029
```

Two additional arguments are used in the test-cases in order to define the range of valid results. These are only needed when you test your code (with the flask-button) or when you submit. You will see the range of valid answers when you run the tests and your code throws an error.

Always submit partial solutions

Do not hardcode, we will change the test values.